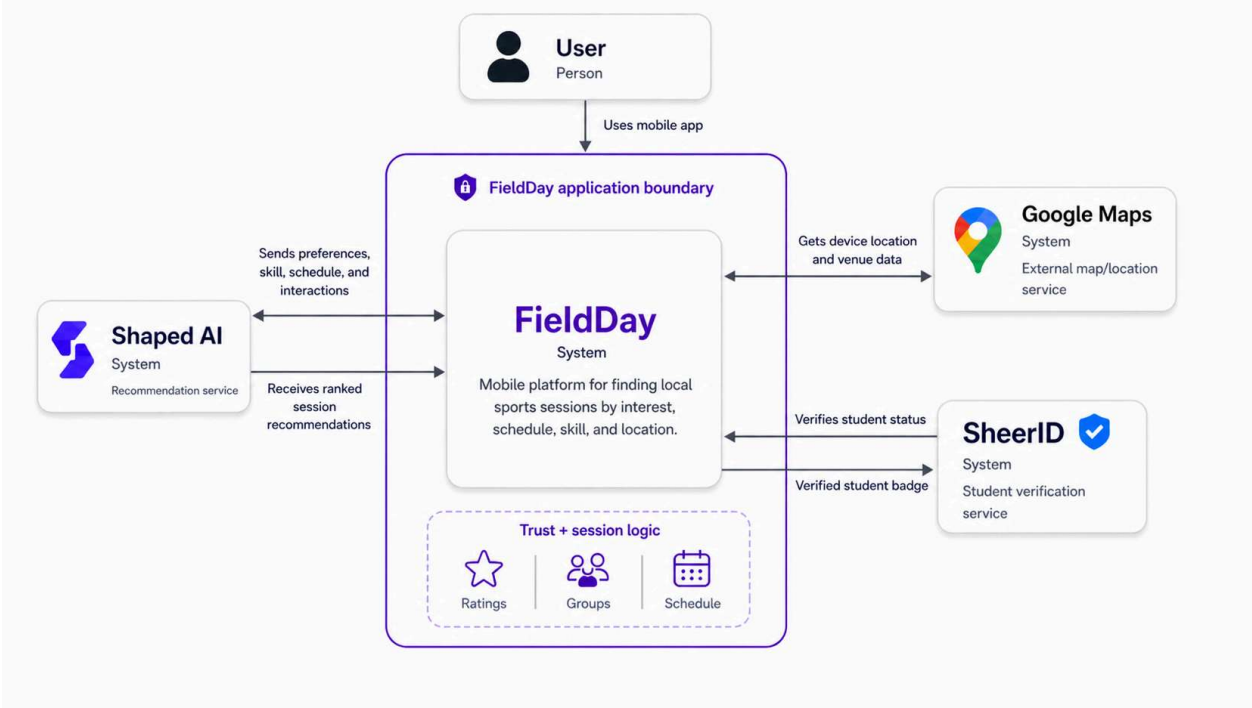
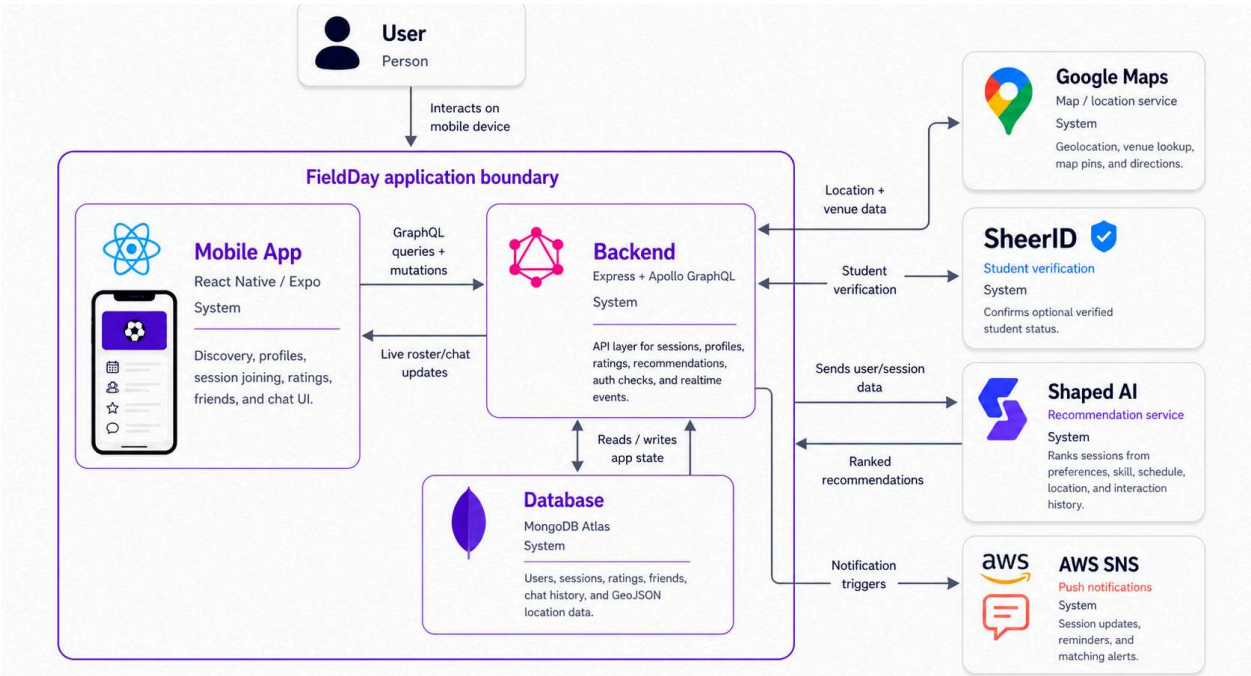
Design Document
Team 4: SOKO

Design Diagrams

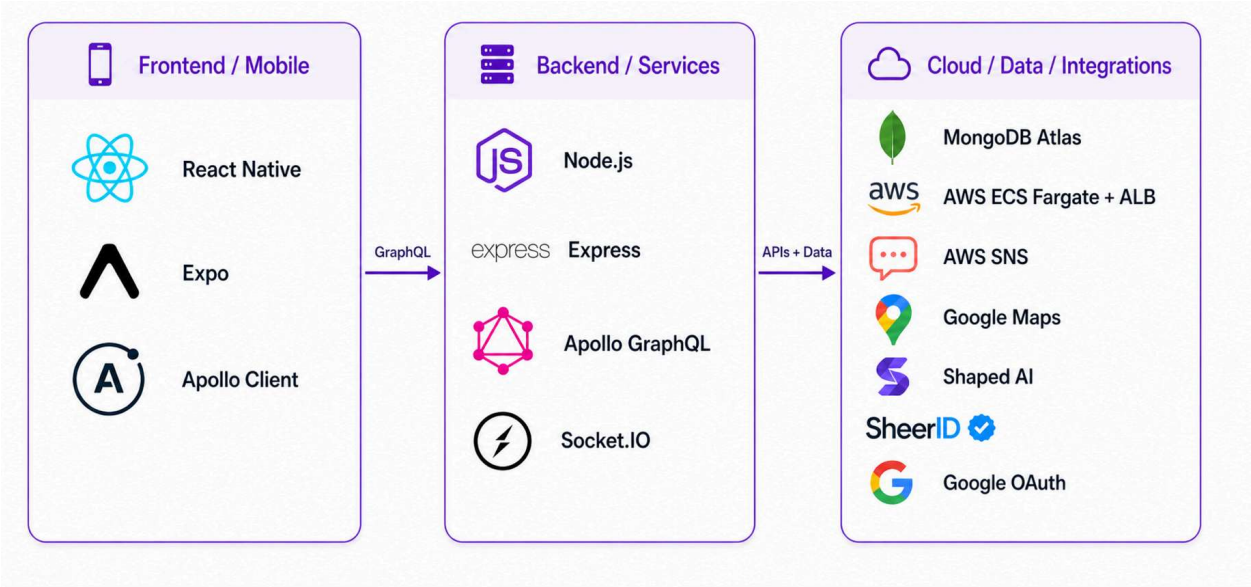
System Context Diagram



Container Diagram



Technology Stack



Figma

Figma Link

https://www.figma.com/design/IKp7VEiHzNek8C2cEWrcAK/Team-4---FieldDay?node-id=3628-993&t=gBWSbXbwl0x1Ks0W-1

Design Trade

Recommendation Algorithm
One of the design alternatives we considered was in replacement of the recommendation algorithm. We considered a forum page that would allow users to reach out and connect to other users/sessions, a search function with filters for users to find sessions, and a list of recent user activity for users to find active users/sessions.

Forum Page
This would essentially be like Reddit where users could advertise themselves and their sessions in a forum. It would likely be chaotic, but would probably yield in a lot of sessions being filled.

Search Function
This would allow users to search by certain criteria to find a session of their choice. They could filter by sport, skill, time, and location.

Recent User Activity
This would simply be a page listing the most recent user activity within a local area. For example, it would include session postings and past sessions so that users could find where and when people often play.

Criteria	Weight	Forum Page	Weighted	Search Function	Weighted	Recent Activity	Weighted	Recommendation Algorithm	Weighted
New user usability	30%	4	1.2	6	1.8	6	1.8	7	2.1
Existing user expansion	10%	6	0.6	7	0.7	9	0.9	9	0.9
Light load on user	20%	3	0.6	2	0.4	5	1	8	1.6
Ability to find similar users	30%	7	2.1	9	2.7	3	0.9	8	2.4
Scalability	10%	5	0.5	6	0.6	6	0.6	8	0.8
Weighted Total			5		6.2		5.2		7.8

Above is the trade off study matrix. The three alternatives are highlighted in red, while the actual recommendation algorithm, highlighted in yellow, is what we ended up deciding upon. One important trade off is the new versus existing user effectiveness. The recommendation algorithm was really good at expanding the existing user base, but slightly weaker at getting new users to find good sessions. However, since it was scalable, convenient, and matched users with sessions well, we ended up going with the recommendation algorithm anyways.

Backend Compute Infrastructure

The alternatives considered for our backend compute infrastructure were:

Design A: AWS EC2 with Elastic Load Balancer (Traditional Server-Based)

The backend application is deployed as a Node.js process on EC2 instances behind an Application Load Balancer (ALB), requiring the team to manage the OS, runtime, and application lifecycle. This approach offers full WebSocket support and no cold starts but imposes a high operational burden.

Design B: AWS Lambda with API Gateway (Serverless)
The backend is decomposed into short-lived, stateless Lambda functions, with API Gateway handling HTTP and a separate WebSocket API for real-time communication. This offers low idle cost and automatic scaling but fundamentally lacks native Socket.IO support.

Design C: AWS ECS Fargate (Container Orchestration)
The application is packaged as a Docker container and deployed as long-running Fargate tasks managed by ECS, which handles the underlying server infrastructure. This solution provides native Socket.IO compatibility and faster scaling than EC2, while eliminating the operational burden of managing the OS.

Trade Study Matrix

Criterion	**Weight**	**EC2**	**Lambda**	**ECS Fargate**
WebSocket Support	0.25	10	3	9
Cost Efficiency	0.2	5	9	6
Horizontal Scalability	0.2	6	6	8
Operational Complexity	0.15	5	5	7
Cold-Start Latency	0.1	9	4	8
Fault Tolerance	0.1	7	9	9
Total	**1.0**	**7.05**	**6.85**	**7.8**

The AWS ECS Fargate solution was selected for our backend infrastructure, achieving the highest total score of 7.8 in our trade study. This decision was largely driven by its superior balance of features and lack of critical weaknesses compared to the alternatives. Specifically, Fargate provides native support for persistent WebSocket connections, a core requirement for real-time chat and roster updates that the Lambda-based serverless model simply could not support. Additionally, Fargate offers better operational efficiency than EC2 by automating OS management and providing integrated fault tolerance, which reduces the maintenance burden on

the team. While Fargate carries a higher idle cost than Lambda, we decided this trade-off was acceptable given our access to educational credits and the significant time saved on infrastructure maintenance.

Backend API Server

Alternative 1: Monolithic GraphQL Server
A single Express.js and Apollo GraphQL backend would manage session creation, user profiles, recommendations, chat, database access, and third-party API integrations in one codebase. This design is simpler to build and maintain, making it a good fit for a small team and fixed project deadline.

Alternative 2: Microservices Architecture
The backend would be divided into separate services, such as User Service, Session Service, Recommendation Service, and Chat Service. This improves scalability and independent service deployment, but it creates more complexity in communication, testing, and deployment.

Alternative 3: Serverless Backend-as-a-Service
The backend would use serverless functions, such as AWS Lambda, to handle API requests only when needed. This option can reduce infrastructure cost and scale automatically, but cold starts and WebSocket support may create issues for real-time chat and session updates.

	Weight	Monolithic GraphQL Server	Microservices	Serverless BaaS
Latency	25%	9 (2.25)	8 (2.00)	6 (1.50)
Development Complexity	20%	10 (2.00)	4 (0.80)	7 (1.40)
Scalability	15%	6 (0.90)	10 (1.50)	9 (1.35)
Scalability	20%	9 (1.80)	6 (1.20)	7 (1.40)
Security	20%	9 (1.80)	7 (1.40)	8 (1.60)
Total Score	100%	8.75	6.9	7.25

The chosen solution is Monolithic GraphQL Server because it had the highest total score and best fits the team's budget, timeline, and technical requirements. This design supports low latency, easier third-party API integration, and simpler implementation of security features such as OAuth, TLS, and role-based access control. Although microservices and serverless designs offer stronger scalability, they introduce extra complexity that could slow development and increase risk. The monolithic approach provides the best trade-off between performance, maintainability, and realistic implementation for this project.

Databases

AD-1: PostgreSQL + PostGIS
It has advanced geographical functionality available natively as well as advanced SQL

features, and as such is suitable for analytics of contained data and customizability for our own usage, both due to the usage of additional extensions and its general nature. It is also self-hosted, and would not incur additional charges as such, however may be more complex to implement, given it requires additional pre-configuration (such as table design).

AD-2: Amazon DocumentDB
An alternative to our current approach but with strong similarities. It features a better cloud integration as well as better scalability and reliability. Additionally, it would natively work with AWS, which is currently our considered option for deployment. The disadvantage, however, is that it would lock us to using AWS and limit our freedom in selecting other components for the project.

AD-3: Neo4j
Features an improved querying due to its node-based approach, with certain categories such as interest and attendance being used as edges to link the nodes together and is as such well-suited for relationship-centric data. That makes it a good choice due to the recommendation system we plan on implementing. It can make use of extensions to implement geospatial querying but does not feature native implementation. On the other hand, it differs from a traditional database approach, and as such can be difficult to implement properly compared to document stores such as AD-2.

Trade study matrix:

Design	Geospatial querying	Cost	Mutability	Scalability	Compat.	Raw Score	Weighted Score
OD	7	10	10	8	7	42	8.6
AD-1	10	10	7	7	9	43	8.6
AD-2	7	6	10	9	8	40	7.85
AD-3	3	10	10	6	5	34	7.3

Given the first alternative design, AD-1 (PostgreSQL + PostGIS) was scored equally to the original design but with one point higher in the raw score, it would be a suitable replacement. On the other hand, it is more complex to use and has less mutability, so it is not a clear improvement over the currently chosen design. As such, it may be worth keeping it in mind should there be issues found with the current design, while going

ahead with what is already planned.